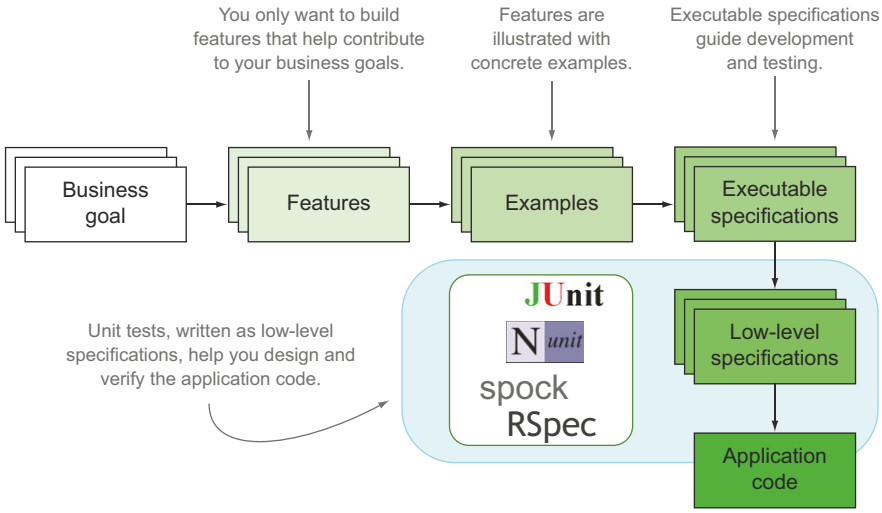


Figure 10.1 In this chapter we'll focus on using BDD practices at the unit-testing level.

- BDD is about writing executable specifications that guide the implementation at all levels of development.
- At a unit-testing level, BDD builds on and extends established TDD practices.
- BDD practitioners use an outside-in approach, using automated acceptance tests and unit tests to drive the implementation of the underlying code.
- You can practice BDD-style unit testing with any tool, but some tools make it easier to write more expressive and more concise unit tests.
- BDD practices at the unit-testing level also help provide living technical documentation of the components and APIs you develop for your application.

Let's start by taking a closer look at the relationship between BDD and TDD.

10.1 BDD, TDD, and unit testing

In this chapter, you'll learn how BDD principles can also help developers write more focused, more effective, more maintainable, and better-documented low-level code, and how unit tests can be used very effectively to express, document, and validate low-level specification and design.

"But isn't that TDD?" I hear you asking. There's often confusion about the distinction between Behavior-Driven Development and Test-Driven Development (TDD). Many developers think of BDD as a technique used for acceptance testing and use "TDD" to refer to lower-level, test-first activities involving unit tests. In fact, things aren't that clear-cut, and the two techniques are deeply intertwined.